

Decision Companion - Backend Architecture Guide

Table of Contents

1. [Architecture Overview](#1-architecture-overview)

1. Architecture Overview

High-Level Architecture

The Decision Companion backend follows a **layered architecture** pattern with clear separation of concerns:

Technology Stack

Component	Technology	Version	Purpose
Framework	Django	5+	Web framework
API	Django REST Framework	3.14+	REST API
Authentication	SimpleJWT	5.3+	JWT tokens
Database	PostgreSQL/SQLite	15+/3	Data storage
CORS	django-cors-headers	4.3+	Cross-origin
Python	Python	3.10+	Runtime

Key Design Principles

1. **Single Responsibility**: Each app handles one domain

2. Project Structure

```
decision_companion/
```

App Responsibilities

App	Responsibility
core	User authentication, settings,
decisions	Decision CRUD, options, factor
feedback	User feedback collection and m
admin_panel	Admin-only analytics and confi

3. Design Patterns

3.1 Model-View-Serializer (MVS) Pattern

Django REST Framework's variation of MVC:

```
# MODEL - Data structure and business rules
class Decision(models.Model):
    title = models.CharField(max_length=200)
    user = models.ForeignKey(User, on_delete=models.CASCADE)

    def can_analyze(self):
        return self.options.count() >= 2

# VIEW - Request/Response handling
class DecisionViewSet(ModelViewSet):
    serializer_class = DecisionSerializer

    def get_queryset(self):
        return Decision.objects.filter(user=self.request.user)

# SERIALIZER - Data transformation
class DecisionSerializer(ModelSerializer):
    class Meta:
        model = Decision
        fields = '__all__'
```

3.2 Service Layer Pattern

Complex business logic is encapsulated in service classes:

```
# services.py
class AIService:
    def __init__(self, decision):
        self.decision = decision

    def analyze_decision(self):
        """Core analysis algorithm"""
        options = self._calculate_option_scores()
        return self._generate_recommendation(options)

    def _calculate_option_scores(self):
        """Internal calculation method"""
        pass
```

3.3 Repository Pattern (via Django ORM)

```
# Custom managers act as repositories
class DecisionManager(models.Manager):
    def for_user(self, user):
        return self.filter(user=user)

    def analyzed(self):
        return self.filter(status='analyzed')

class Decision(models.Model):
    objects = DecisionManager()
```

3.4 Mixin Pattern

```
# Reusable behavior through mixins
class OwnerQuerySetMixin:
    def get_queryset(self):
        return super().get_queryset().filter(user=self.request.user)

class DecisionViewSet(OwnerQuerySetMixin, ModelViewSet):
    pass
```

4. Configuration

4.1 Settings Structure

```
# decision_companion/settings.py

# Security
SECRET_KEY = 'your-secret-key'
DEBUG = True # False in production
ALLOWED_HOSTS = []

# Application definition
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',

    # Third-party
    'rest_framework',
    'rest_framework_simplejwt',
    'rest_framework_simplejwt.token_blacklist',
    'corsheaders',

    # Local apps
    'apps.core',
    'apps.decisions',
    'apps.feedback',
    'apps.admin_panel',
]

# Custom user model
AUTH_USER_MODEL = 'core.User'

# REST Framework configuration
REST_FRAMEWORK = {
    'DEFAULT_AUTHENTICATION_CLASSES': [
```

```
        'rest_framework_simplejwt.authentication.JWTAuthentication',
    ],
    'DEFAULT_PERMISSION_CLASSES': [
        'rest_framework.permissions.IsAuthenticated',
    ],
    'DEFAULT_PAGINATION_CLASS': 'rest_framework.pagination.PageNumberPagination',
    'PAGE_SIZE': 20,
}

# JWT Configuration
SIMPLE_JWT = {
    'ACCESS_TOKEN_LIFETIME': timedelta(minutes=5),
    'REFRESH_TOKEN_LIFETIME': timedelta(days=1),
    'ROTATE_REFRESH_TOKENS': False,
    'BLACKLIST_AFTER_ROTATION': True,
    'AUTH_HEADER_TYPES': ('Bearer',),
}

# Database
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.sqlite3',
        'NAME': BASE_DIR / 'db.sqlite3',
    }
}
```

4.2 Environment Variables

For production, use environment variables:

```
import os
from dotenv import load_dotenv

load_dotenv()

SECRET_KEY = os.getenv('SECRET_KEY')
DEBUG = os.getenv('DEBUG', 'False') == 'True'
DATABASE_URL = os.getenv('DATABASE_URL')
```

5. Authentication System

5.1 Custom User Model

```
class User(AbstractBaseUser, PermissionsMixin):
    """Custom user with email authentication"""

    ROLE_CHOICES = [
        ('USER', 'User'),
        ('ADMIN', 'Admin'),
```

```
]

id = models.UUIDField(primary_key=True, default=uuid.uuid4)
email = models.EmailField(unique=True)
name = models.CharField(max_length=255)
role = models.CharField(max_length=10, choices=ROLE_CHOICES, default='USER')
is_guest = models.BooleanField(default=False)
is_active = models.BooleanField(default=True)
is_staff = models.BooleanField(default=False)

USERNAME_FIELD = 'email'
REQUIRED_FIELDS = ['name']

objects = UserManager()
```

5.2 JWT Authentication Flow

1. User sends credentials to /api/v1/auth/login
2. Server validates credentials
3. Server generates access + refresh tokens
4. Client stores tokens
5. Client includes "Authorization: Bearer <access_token>" in requests
6. Server validates token on each request
7. When access token expires, client uses refresh token
8. On logout, refresh token is blacklisted

5.3 Permission Classes

```
# apps/core/permissions.py

class IsAdminUser(BasePermission):
    """Only allow admin users"""
    def has_permission(self, request, view):
        return request.user.role == 'ADMIN'

class IsOwnerOrAdmin(BasePermission):
    """Allow owners or admins"""
    def has_object_permission(self, request, view, obj):
        if request.user.role == 'ADMIN':
            return True
        return obj.user == request.user
```

6. Database Design

6.1 Model Relationships

6.2 Model Best Practices

```
class BaseModel(models.Model):
```

```
"""Abstract base model with common fields"""
id = models.UUIDField(primary_key=True, default=uuid.uuid4, editable=False)
created_at = models.DateTimeField(auto_now_add=True)
updated_at = models.DateTimeField(auto_now=True)

class Meta:
    abstract = True

class Decision(BaseModel):
    """Concrete model inheriting base fields"""
    user = models.ForeignKey(
        settings.AUTH_USER_MODEL,
        on_delete=models.CASCADE,
        related_name='decisions'
    )
    title = models.CharField(max_length=200)

    class Meta:
        ordering = ['-created_at']
        indexes = [
            models.Index(fields=['user', 'status']),
            models.Index(fields=['created_at']),
        ]
```

6.3 QuerySet Optimization

```
# Avoid N+1 queries with select_related and prefetch_related
decisions = Decision.objects.select_related('user').prefetch_related(
    'options',
    'factors',
    'factors__scores',
    'result'
).filter(user=user)
```

7. API Architecture

7.1 URL Structure

```
/api/v1/
```

7.2 ViewSet Pattern

```
class DecisionViewSet(ModelViewSet):
    """
    ViewSet provides CRUD operations automatically:
    - list()      GET /decisions/
    - create()    POST /decisions/
    - retrieve()  GET /decisions/{id}/
    - update()    PUT /decisions/{id}/
    - partial_update() PATCH /decisions/{id}/
    """
```

```
- destroy() DELETE /decisions/{id}/
"""
serializer_class = DecisionSerializer
permission_classes = [IsAuthenticated]

def get_queryset(self):
    return Decision.objects.filter(user=self.request.user)

def perform_create(self, serializer):
    serializer.save(user=self.request.user)

@action(detail=True, methods=['post'], url_path='ai/analyze')
def analyze(self, request, pk=None):
    """Custom action for AI analysis"""
    decision = self.get_object()
    # ... analysis logic
```

7.3 Serializer Patterns

```
# Nested serialization
class DecisionDetailSerializer(ModelSerializer):
    options = DecisionOptionSerializer(many=True, read_only=True)
    factors = FactorSerializer(many=True, read_only=True)
    result = ResultSerializer(read_only=True)

    class Meta:
        model = Decision
        fields = '__all__'

# Write operations with nested data
class DecisionCreateSerializer(ModelSerializer):
    options = DecisionOptionSerializer(many=True, required=False)

    def create(self, validated_data):
        options_data = validated_data.pop('options', [])
        decision = Decision.objects.create(**validated_data)
        for option_data in options_data:
            DecisionOption.objects.create(decision=decision, **option_data)
        return decision
```

8. Business Logic

8.1 Service Layer

The AI analysis is implemented in a dedicated service class:

```
# apps/decisions/services.py

class AIService:
```

```
"""
Handles all AI-related business logic for decision analysis.

Responsibilities:
- Calculate weighted scores for options
- Generate recommendations
- Calculate confidence levels
- Suggest relevant factors
"""

def __init__(self, decision):
    self.decision = decision
    self._load_ai_parameters()

def _load_ai_parameters(self):
    """Load configurable parameters from database"""
    params = AIParameters.objects.all()
    self.params = {p.key: p.typed_value for p in params}

def analyze_decision(self):
    """
    Main analysis algorithm.

    Returns:
        Result object with recommendation
    """
    option_scores = self._calculate_option_scores()
    recommended = self._determine_winner(option_scores)
    confidence = self._calculate_confidence(option_scores)

    return Result.objects.create(
        decision=self.decision,
        recommended_option=recommended,
        confidence_level=confidence,
        explanation=self._generate_explanation(recommended),
        analysis_data={'scores': option_scores}
    )
```

8.2 Algorithm Implementation

```
def _calculate_option_scores(self):
    """
    Calculate weighted scores for each option.

    Formula:
```

9. Error Handling

9.1 Custom Exceptions


```
# apps/core/exceptions.py

class ApplicationException(Exception):
    """Base exception for application errors"""
    default_message = "An error occurred"
    default_code = "error"

    def __init__(self, message=None, code=None):
        self.message = message or self.default_message
        self.code = code or self.default_code
        super().__init__(self.message)

class ValidationException(ApplicationException):
    default_message = "Validation failed"
    default_code = "validation_error"

class NotEnoughDataException(ApplicationException):
    default_message = "Not enough data for analysis"
    default_code = "insufficient_data"
```

9.2 Exception Handler

```
# Custom exception handler in views or settings
from rest_framework.views import exception_handler

def custom_exception_handler(exc, context):
    response = exception_handler(exc, context)

    if response is not None:
        response.data = {
            'success': False,
            'error': {
                'code': getattr(exc, 'code', 'error'),
                'message': str(exc.detail) if hasattr(exc, 'detail') else str(exc),
            }
        }

    return response
```

9.3 Response Format

```
# Consistent response structure

# Success response
{
    "success": True,
    "data": {
        "id": "uuid",
        "title": "My Decision"
    }
}
```

```
# Error response
{
  "success": False,
  "error": {
    "code": "validation_error",
    "message": "Title is required",
    "details": {
      "title": ["This field is required."]
    }
  }
}
```

10. Testing Strategy

10.1 Test Structure

tests/

10.2 Test Examples

```
# tests/test_views.py
from rest_framework.test import APITestCase
from rest_framework import status

class DecisionViewSetTest(APITestCase):
    def setUp(self):
        self.user = User.objects.create_user(
            email='test@example.com',
            password='testpass123',
            name='Test User'
        )
        self.client.force_authenticate(user=self.user)

    def test_create_decision(self):
        data = {'title': 'Test Decision', 'description': 'Test'}
        response = self.client.post('/api/v1/decisions/', data)

        self.assertEqual(response.status_code, status.HTTP_201_CREATED)
        self.assertEqual(response.data['data']['title'], 'Test Decision')

    def test_list_own_decisions_only(self):
        other_user = User.objects.create_user(
            email='other@example.com',
            password='pass',
            name='Other'
        )
        Decision.objects.create(user=other_user, title='Other Decision')
        Decision.objects.create(user=self.user, title='My Decision')
```

```
response = self.client.get('/api/v1/decisions/')

self.assertEqual(len(response.data['results']), 1)
self.assertEqual(response.data['results'][0]['title'], 'My Decision')
```

10.3 Running Tests

```
# Run all tests
python manage.py test

# Run specific app tests
python manage.py test apps.decisions

# Run with coverage
coverage run manage.py test
coverage report
```

11. Deployment

11.1 Production Checklist

- [] Set `DEBUG = False`

• •

```
# Dockerfile
FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

RUN python manage.py collectstatic --noinput

EXPOSE 8000

CMD ["gunicorn", "decision_companion.wsgi:application", "--bind", "0.0.0.0:8000"]
```

```
# docker-compose.yml
version: '3.8'

services:
  web:
    build: .
    ports:
      - "8000:8000"
    environment:
```

```
- DATABASE_URL=postgres://user:pass@db:5432/decision_companion
- SECRET_KEY=${SECRET_KEY}
depends_on:
  - db

db:
  image: postgres:15
  volumes:
    - postgres_data:/var/lib/postgresql/data
  environment:
    - POSTGRES_DB=decision_companion
    - POSTGRES_USER=user
    - POSTGRES_PASSWORD=pass

volumes:
  postgres_data:
```

11.3 Environment Configuration

```
# .env.production
DEBUG=False
SECRET_KEY=your-production-secret-key
DATABASE_URL=postgres://user:pass@localhost:5432/decision_companion
ALLOWED_HOSTS=api.decisioncompanion.com
CORS_ALLOWED_ORIGINS=https://app.decisioncompanion.com
```

12. Best Practices

12.1 Code Style

```
# Use type hints
def calculate_score(factor: Factor, option: DecisionOption) -> float:
    pass

# Use docstrings
def analyze_decision(self) -> Result:
    """
    Analyze decision and generate recommendation.

    Returns:
        Result: Analysis result with recommendation

    Raises:
        NotEnoughDataException: If fewer than 2 options exist
    """
    pass

# Use constants
class DecisionStatus:
```

```
DRAFT = 'draft'
ANALYZED = 'analyzed'
ARCHIVED = 'archived'
```

12.2 Security Best Practices

1. ****Always validate input data**** using serializers

12.3

1. ****Use `select\_related`/`prefetch\_related`**** to avoid N+1 queries

12.4

1. ****Use consistent response format****